

CS 415

Software Engineering

Project Milestone 1 – Build It

StudyFlow

Smart Academic Workload and Deadline Management System

Team Members

Eunjoo Jung

Ejung7@crimson.ua.edu

Errin James

Emjames1@crimson.ua.edu

Shared GitHub Repository

<http://github.com/hihi0305/studyflow>

Submission Date

3 October 2026

Milestone 1 Overview

Milestone 1 advances StudyFlow from the planning and requirements stage established in Milestone 0 to a working end-to-end minimum viable product (MVP). This milestone transforms the initial project design into a functional web application that supports the core academic-management workflow for an authenticated student user.

The Milestone 1 MVP focuses on the essential workflow of user registration and authentication, course management, academic task management, task progress and status tracking, persistent data storage, and a centralized dashboard. The end-to-end workflow allows a student to register an account, log in, create and manage courses, create and manage academic tasks, update task progress or status, and view saved information through the dashboard.

StudyFlow continues to follow the incremental development approach established in Milestone 0. Milestone 1 concentrates on establishing a stable and reproducible core application before more advanced functionality is introduced. Features such as Task Attention Level calculation, advanced search and filtering, workload analytics, notifications, calendar integration, and AI-assisted functionality remain outside the current MVP scope and are deferred to later milestones.

The completed MVP was verified through an end-to-end workflow covering registration, login, course management, academic task management, progress and status updates, dashboard behavior, logout, re-login, persistent data storage, and user-specific data isolation. The Milestone 1 implementation and documentation were reviewed together so that this report reflects the functionality that can be run and verified from the submitted repository.

1 Updated Product Brief and MVP Scope

1.1 Updated Product Brief

StudyFlow is a responsive web-based academic planning application designed to help college and university students organize courses, academic tasks, deadlines, workload information, and task progress within a centralized system. The application addresses the difficulty students often experience when academic responsibilities are distributed across course-management systems, calendars, personal notes, and other tools.

The Milestone 1 version of StudyFlow establishes the application's core academic-management platform. The MVP focuses on authenticated, user-specific management of courses and academic tasks. Students can create and manage courses identified by course number, create and manage academic tasks, record information such as due dates, task type, user-defined priority, estimated work hours, progress, and status, and view their current academic responsibilities through a centralized dashboard.

For Milestone 1, course records are intentionally limited to a course number. Additional course metadata such as semester, section, and full course title are deferred to later work so that the MVP remains focused on the required end-to-end workflow.

StudyFlow is implemented as a client-server web application using a React frontend, a Node.js and Express backend, a REST/JSON API, and PostgreSQL for persistent relational data storage. Authentication and authorization protect account-specific academic information and prevent users from accessing another user's private data.

The broader StudyFlow vision remains unchanged from Milestone 0. However, Milestone 1 intentionally limits the product scope to a stable end-to-end MVP. More advanced workload-analysis and productivity features are deferred until after the core application workflow has been implemented and verified.

1.2 MVP Scope

The Milestone 1 MVP is defined around one complete academic-management workflow. An authenticated student can create an account, log in, create and manage courses, create

and manage academic tasks, update task progress and status, and view saved academic information through the dashboard.

The MVP is intentionally limited to functionality necessary to support this core workflow. The purpose of this boundary is to reduce unnecessary implementation risk and ensure that the team produces a stable, reproducible, and verifiable application before adding more advanced functionality.

The Milestone 1 MVP includes user registration, login and logout, course creation and management, academic task creation and management, task type selection, due-date entry, user-defined priority, estimated work-hour entry, progress and status tracking, persistent database storage, a basic dashboard, and user-specific access control.

1.3 Features Included in the Milestone 1 MVP

Feature Area	Milestone 1 Scope
User Accounts	User registration, login, and logout
Authentication	Authentication and protection of authenticated functionality
User Data Protection	Users may access and modify only their own academic data
Course Management	Create, view, edit, and delete courses
Academic Task Management	Create, view, edit, and delete academic tasks
Task Organization	Associate academic tasks with courses
Task Details	Store title, task type, due date, user-defined priority, and estimated work hours
Progress Tracking	Record progress from 0–100%
Status Tracking	Support Not Started, In Progress, and Completed states
Dashboard	Display the authenticated user's academic tasks and important task information
Persistence	Store user, course, and task data in PostgreSQL
Web Interface	Provide a usable responsive browser-based interface

1.4 Features Deferred Beyond Milestone 1

Deferred Feature	Planned Timing / Reason
Task Attention Level calculation	Planned after the core MVP is stable

Deferred Feature	Planned Timing / Reason
Advanced search and filtering	Planned for a later iteration
Workload recommendations	Requires additional analysis logic beyond the core MVP
Calendar or timeline view	Enhancement beyond the core MVP workflow
Academic workload analytics	Requires additional aggregation and visualization
Reminder notifications	Not required for the initial end-to-end workflow
Calendar synchronization	Requires external-service integration
AI-assisted task decomposition	Advanced feature outside the core MVP

1.5 Scope Boundaries

Milestone 1 is considered successful when the core StudyFlow workflow operates from end to end using persistent application data. The milestone does not require the full long-term StudyFlow feature set.

The scope boundary is therefore defined around reliable academic data management rather than advanced prioritization, analytics, external integrations, or intelligent recommendations. This distinction allows the team to establish authentication, data ownership, database relationships, API communication, and the primary user interface before introducing features that depend on those foundations.

Small implementation-level adjustments were made where necessary, but significant scope expansion was avoided unless required to complete the core MVP workflow.

2 Development Process Model

2.1 Development Approach

StudyFlow follows an incremental Agile development approach. The application is developed through working project milestones, with each milestone extending the functionality completed in the previous version.

This approach was selected because StudyFlow is a semester-long software project in which requirements, interface details, and implementation decisions may require small adjustments as development progresses. Incremental development allows the team to evaluate design decisions early, integrate working functionality frequently, and maintain focus on a usable end-to-end application.

For Milestone 1, development was prioritized around the core MVP workflow.

Advanced features were intentionally deferred so that authentication, course management, academic task management, progress tracking, persistence, and dashboard functionality could be implemented and verified first.

2.2 Iterative Work Process

Development was organized into short iterative work cycles of approximately one week when practical. At the beginning of each work cycle, the team reviewed the backlog and selected prioritized implementation or documentation tasks based on Milestone 1 requirements and the current state of the application.

Completed work was reviewed and integrated incrementally, while unfinished work and upcoming priorities were adjusted when implementation or integration issues required changes. This lightweight process was appropriate for a two-person course project because it provided regular checkpoints without requiring a formal large-team Scrum process.

2.3 GitHub Workflow and Collaboration

GitHub served as the primary record of Milestone 1 development activity. Issues were

used to track meaningful implementation, documentation, design, integration, verification, and defect-related work. Significant code and documentation changes were normally developed through short-lived branches, small descriptive commits, and Pull Requests when practical.

The main branch represented the stable integrated project version. Important requests, decisions, work handoffs, implementation updates, and verification results were documented through GitHub Issues, commits, Pull Requests, or reviews when appropriate. If an important discussion occurred outside GitHub, the resulting project decision or action was summarized in GitHub when practical.

The shared Google Doc was used as the primary collaborative writing environment for the report, while GitHub preserved major documentation checkpoints and section-level progress. The project's Definition of Done and acceptance criteria are described separately in Section 3.4.

3 Prioritized Product Backlog and Definition of Done

3.1 Backlog Update for Milestone 1

The StudyFlow product backlog was originally defined during Milestone 0 and was reviewed for Milestone 1 against the required end-to-end MVP scope. High-priority stories supporting authentication, course management, academic task management, dashboard visibility, and progress tracking were selected as the core implementation scope.

US-01 through US-07 were completed as part of the Milestone 1 MVP. US-08 through US-12 remain valid backlog items but were intentionally deferred beyond Milestone 1.

3.2 Prioritized Product Backlog

US-01 through US-07 formed the core Milestone 1 backlog and were completed as part of the working MVP. Together, these stories support the required end-to-end workflow. US-08 through US-12 remain valid product requirements but were intentionally deferred beyond Milestone 1.

The original Milestone 0 backlog assigned High priority to US-01 through US-07, Medium priority to US-08 through US-10, and Low priority to US-11 through US-12.

ID	User Story	Priority	Milestone 1 Plan	Current Status
US – 01	As a new user, I want to create an account so that I can securely access my academic information.	High	Core MVP	Done
US – 02	As a registered user, I want to log in and log out securely so that my academic data remains private.	High	Core MVP	Done
US – 03	As a student, I want to create and manage courses so that I can organize tasks by course.	High	Core MVP	Done
US – 04	As a student, I want to create academic tasks so that I can track assignments,	High	Core MVP	Done

ID	User Story	Priority	Milestone 1 Plan	Current Status
	exams, and projects.			
US – 05	As a student, I want to edit and delete tasks so that my academic information remains accurate.	High	Core MVP	Done
US – 06	As a student, I want to view my active tasks on a dashboard so that I can quickly understand my workload.	High	Core MVP	Done
US – 07	As a student, I want to record task progress so that I know how much work remains.	High	Core MVP	Done
US – 08	As a student, I want StudyFlow to assign Task Attention Levels to my active tasks so that I can decide what to work on first.	Medium	Deferred	Deferred
US – 09	As a student, I want to search and filter tasks so that I can quickly find relevant academic work.	Medium	Deferred	Deferred
US – 10	As a student, I want to view workload information so that I can better plan upcoming study time.	Medium	Deferred	Deferred
US – 11	As a student, I want a calendar or timeline view so that I can understand upcoming deadlines visually.	Low	Deferred	Deferred
US – 12	As a student, I want StudyFlow to suggest how to break down a large task so that I can plan complex assignments.	Low	Deferred	Deferred

3.3 Backlog Status Definitions

StudyFlow uses the following status values to describe the current state of backlog work:

Status	Meaning
Backlog	The work has been identified but implementation or documentation has not yet started.
In Progress	Meaningful work on the item is currently underway.
In Review	The primary work is complete and is awaiting review, verification, or integration.
Done	The item satisfies the project's Definition of Done and has been integrated into the appropriate shared project version.
Deferred	The requirement remains part of the product backlog but is intentionally

Status	Meaning
	outside the current milestone scope.

Priority describes the relative importance of a backlog item, while status describes its current state of completion.

3.4 Definition of Done

A Milestone 1 backlog item is considered Done when:

- The agreed acceptance criteria are satisfied.
- The implementation or documentation performs the intended behavior successfully.
- Relevant verification is completed and existing tests continue to pass when applicable.
- Required documentation is updated when setup, behavior, interfaces, or project decisions change.
- No known critical defect prevents demonstration or verification.
- The work is integrated into the appropriate shared project version.
- The related GitHub Issue reflects the current completion status and significant verification evidence when appropriate.

4 Use Cases

4.1 Main Use Cases

The following use cases describe the primary user interactions required for the Milestone 1 core MVP. They are based on the high-priority backlog items defined for registration and authentication, course management, academic task management, and dashboard-based progress tracking.

4.2 Registration and Login

Primary Actor: Student

Related User Stories: US-01 Account Registration, US-02 Login/Logout

Preconditions:

- The user has access to the StudyFlow web application.
- Registration requires an email address that is not already associated with existing account.
- Login requires an existing StudyFlow account.

Main Success Flow:

- a. A new user opens the registration page.
- b. The user enters a valid name, email address, password, and password confirmation.
- c. StudyFlow validates the submitted information.
- d. StudyFlow creates the user account.
- e. The registered user opens the login page.
- f. The user enters the registered email address and password.
- g. StudyFlow validates the credentials.
- h. The user is authenticated and granted access to the dashboard, courses, and academic tasks.
- i. When finished, the authenticated user can log out.

Alternative and Error Flows:

- If a required registration field is missing or invalid, StudyFlow displays an

understandable validation message.

- If the email address is already registered, StudyFlow rejects the duplicate account request.
- If the login credentials are invalid, StudyFlow denies access and displays an understandable error message.
- If the password confirmation does not match the password, the registration form is not submitted.
- An unauthenticated user cannot access protected user data.

Postconditions:

- A successfully registered account is stored persistently.
- After successful login, the user can access only their own academic information.
- After logout, authenticated access ends until the user logs in again.

4.3 Course Management

Primary Actor: Authenticated Student

Related User Story: US-03 Course Management

Preconditions:

- The user is registered and authenticated.

Main Success Flow:

- a. The authenticated user opens the course-management interface.
- b. StudyFlow displays the courses belonging to the user.
- c. The user chooses to add a new course.
- d. The user enters the required course number.
- e. StudyFlow validates and saves the course.
- f. The new course appears in the user's course list.
- g. The course becomes available when the user creates or edits an academic task.
- h. The user may select an existing course and edit its course number.
- i. The user may delete one of their existing courses when appropriate.

Alternative and Error Flows:

- If the required course number is missing or invalid, StudyFlow rejects the operation and displays an understandable validation message.
- If the user cancels an operation, the existing course information remains unchanged.
- A user cannot view, modify, or delete a course belonging to another account.

Postconditions:

- Created or edited course information is stored persistently.
- Only the authenticated user's courses are accessible to that user.
- Deleted courses are removed from the authenticated user's course list. Academic tasks are not deleted solely because an associated course is removed.

4.4 Academic Task Management

Primary Actor: Authenticated Student

Related User Stories: US-04 Create Academic Task, US-05 Edit/Delete Task

Preconditions:

- The user is authenticated.
- The user has access to the academic-task interface.
- A user-owned course is available when the task is associated with a course.

Main Success Flow:

- a. The authenticated user chooses to create a new academic task.
- b. StudyFlow presents the task-entry interface.
- c. The user enters a task title and due date.
- d. The user may associate the task with one of their courses.
- e. The user may enter additional information, including task type, user-defined priority, estimated work hours, progress, and status.
- f. StudyFlow validates the task information.
- g. StudyFlow saves the task.
- h. The saved task becomes available in the user's task list or dashboard.
- i. The user may later edit the task information.
- j. StudyFlow validates and saves the updated information.

- k. The user may delete the task when it is no longer needed.

Alternative and Error Flows:

- A task without a valid title or due date is rejected.
- If the selected course does not belong to the authenticated user, the request is rejected.
- A user cannot modify or delete another user's academic task.
- If an edit or delete operation is canceled, the existing task remains unchanged.

Postconditions:

- Created or edited tasks are stored persistently.
- Updated task information is reflected in the appropriate StudyFlow interface.
- Users can access and modify only their own academic tasks.

4.5 Dashboard and Progress Workflow

Primary Actor: Authenticated Student

Related User Stories: US-06 Dashboard, US-07 Progress

Preconditions:

- The user is authenticated.
- The user may have one or more previously saved academic tasks.

Main Success Flow:

- a. The authenticated user opens the StudyFlow dashboard.
- b. StudyFlow retrieves the academic tasks associated with the authenticated user's account.
- c. The dashboard displays the user's academic tasks and summary information.
- d. Important task information is displayed, including the task title, associated course when available, due date, user-defined priority, progress, and status.
- e. When a task requires a progress or status change, the user updates the task through the academic-task management interface.
- f. The user enters a progress value between 0% and 100% and/or changes the task status.

- g. StudyFlow validates and saves the update.
- h. The dashboard reflects the updated task information and summary counts.
- i. When a task is marked as Completed, its progress is automatically set to 100%.
- j. Completed tasks are identified by their Completed status.

Alternative and Error Flows:

- If the user has no academic tasks, the dashboard displays an appropriate empty state.
- Progress values outside the valid 0–100% range are rejected.
- Invalid updates are not saved and an understandable error message is displayed.
- Tasks belonging to other users are never displayed through the authenticated user's dashboard.

Postconditions:

- Valid progress and status updates are stored persistently.
- The dashboard reflects the current saved state of the user's academic tasks.
- A task marked as Completed has a progress value of 100%.

4.6 Use Case Traceability

Use Case	Related Backlog Items	Purpose
Registration and Login	US-01, US-02	Establish authenticated user access
Course Management	US-03	Organize academic work by course
Academic Task Management	US-04, US-05	Create and maintain academic responsibilities
Dashboard and Progress Workflow	US-06, US-07	View tasks and track progress/status

5 Analysis Model

StudyFlow's analysis model focuses on the core data and interactions required for the Milestone 1 MVP. The primary entities are User, Course, and AcademicTask. These entities represent the main information that must be stored and managed by the system.

A User represents a registered student who can authenticate and access only their own academic data. Each user may create and manage multiple courses and academic tasks.

A Course represents a course created by the authenticated user. For Milestone 1, each Course stores a course number and provides an organizational relationship for academic tasks. Each course belongs to one user and may contain multiple academic tasks.

An AcademicTask represents an assignment, exam, project, or other academic responsibility. Each task belongs to a user and may be associated with one of that user's courses. An AcademicTask stores a title and due date and may also store a description, task type, user-defined priority, estimated work hours, progress percentage, status, and an optional course association.

The primary relationships in the analysis model are:

User 1 — * Course

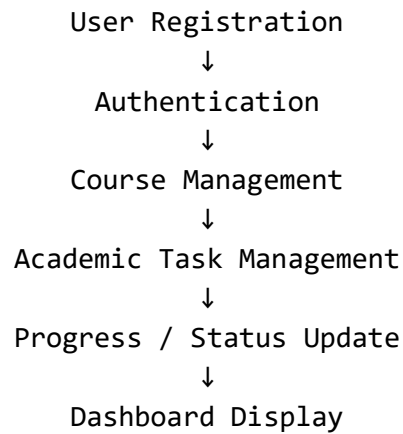
User 1 — * AcademicTask

Course 1 — 0..* AcademicTask

An AcademicTask may be associated with zero or one Course.

This means that one user may own multiple courses and tasks, while each course and task is associated with a specific user. A course may contain multiple academic tasks, while an academic task does not require a course association.

The main system behavior follows the core Milestone 1 workflow:



After authentication, the user can create and manage courses, create academic tasks with or without an optional course association, update task progress and status, and view current task information through the dashboard. The system must preserve user-specific ownership so that one user cannot view or modify another user's courses or tasks.

This analysis model provides the foundation for the system design, database relationships, API design, and component design described in the following sections.

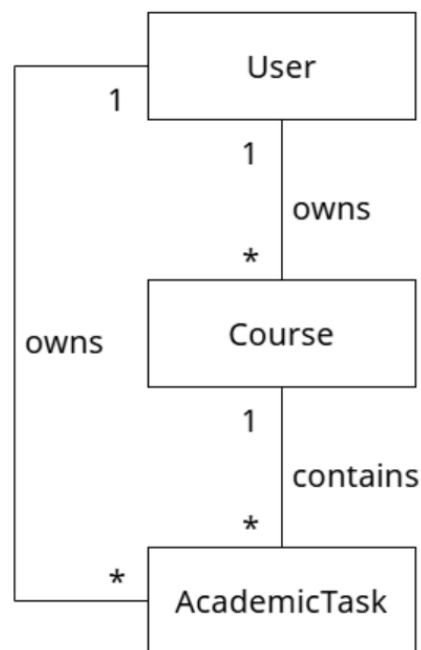


Figure 5.1. Analysis model relationship diagram.

6 System Design

6.1 Design Classes

The StudyFlow system design is centered on three core domain classes: User, Course, and AcademicTask. These classes correspond directly to the main data entities identified in the analysis model.

The User class represents a registered student and stores account-related information required for authentication and user-specific data ownership. A user can own multiple courses and academic tasks.

The Course class represents a course created by an authenticated user. For Milestone 1, it stores the course number and provides an organizational relationship between the user and related academic tasks.

The AcademicTask class represents an assignment, exam, project, or other academic responsibility. A task belongs to a user and may also be associated with a course. It stores information required for task management, such as title, description, due date, task type, priority, estimated work time, progress percentage, and status.

The primary design relationships are:

User 1 — * Course

User 1 — * AcademicTask

Course 1 — 0..* AcademicTask

An AcademicTask may be associated with zero or one Course.

These relationships enforce user-specific ownership while allowing tasks to be organized by course.

Class	Main Responsibility	Example Data
User	Authentication and ownership of user-specific data	id, name, email, password hash

Class	Main Responsibility	Example Data
Course	Stores and organizes course information	id, courseNumber, userId
AcademicTask	Stores academic task details and progress	id, title, description, dueDate, taskType, priority, estimatedHours, progress, status, courseId, userId

6.2 API / Interface Contract

The StudyFlow frontend communicates with the backend through a REST-style JSON API. The API provides the operations required for authentication, course management, academic task management, and dashboard data retrieval.

A simplified Milestone 1 API contract is shown below.

Method	Endpoint	Purpose
POST	/api/auth/register	Create a new user account
POST	/api/auth/login	Authenticate an existing user
GET	/api/auth/me	Retrieve the authenticated user's account information
GET	/api/courses	Retrieve the authenticated user's courses
POST	/api/courses	Create a new course
PUT	/api/courses/:id	Update an existing course
DELETE	/api/courses/:id	Delete a course
GET	/api/tasks	Retrieve the authenticated user's academic tasks
POST	/api/tasks	Create a new academic task
PUT	/api/tasks/:id	Update an existing academic task
DELETE	/api/tasks/:id	Delete an academic task
GET	/api/dashboard	Retrieve data required for the user dashboard

Protected endpoints require a valid JWT in the Authorization header. The backend identifies the authenticated user and restricts course, task, and dashboard data to that user's account.

6.3 Module Boundaries

StudyFlow follows a three-layer structure consisting of the frontend, backend, and database modules.

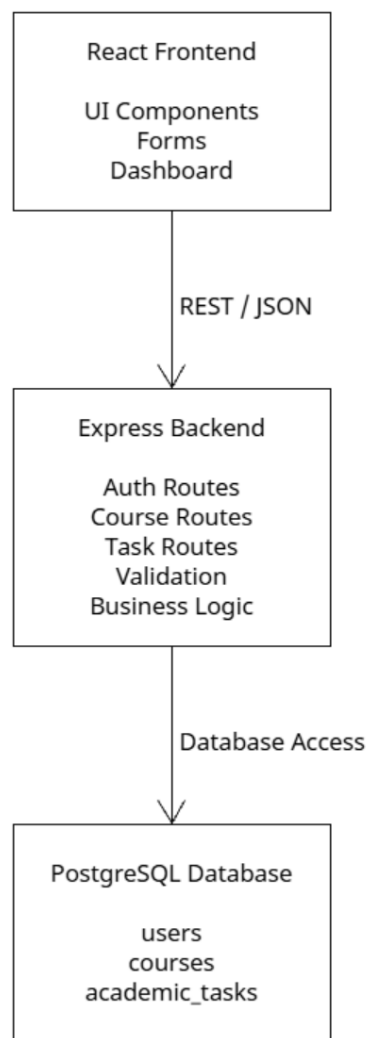


Figure 6.1. Module-boundary diagram.

The frontend module is responsible for presenting the user interface, collecting user input, and displaying data returned by the backend.

The backend module is responsible for authentication, request validation, business logic, user-specific access control, and communication with the database.

The database module is responsible for persistent storage of users, courses, and academic tasks, including task progress and status data.

These module boundaries separate presentation, application logic, and persistent data responsibilities, making the system easier to develop, test, and maintain.

Together, the design classes, API contract, and module boundaries define how the StudyFlow MVP is organized internally. This design supports the core Milestone 1 workflow while keeping the frontend, application logic, and persistent data responsibilities clearly separated.

7 Architecture Decision Record

7.1 Decision

StudyFlow uses a separated client-server architecture consisting of a React frontend, a Node.js/Express backend, and a PostgreSQL database. The frontend communicates with the backend through a REST/JSON API. Authentication and user-specific data access are handled by the backend, with JSON Web Tokens used to authenticate protected requests.

7.2 Context

StudyFlow requires multiple users to create and manage their own courses and academic tasks while keeping each user's data isolated. The application also requires persistent data storage and a clear separation between the user interface, application logic, and database access.

A purely client-side implementation would not provide reliable persistence, secure authentication, or server-side ownership enforcement. Therefore, the application required a backend service responsible for authentication, validation, PostgreSQL database operations, and user-specific ownership enforcement.

7.3 Rationale

The client-server architecture was selected because it separates presentation concerns from application and data-management logic. React is responsible for the user interface, while Express handles API requests, validation, authentication, and business logic. PostgreSQL provides persistent relational storage for users, courses, and academic tasks.

This structure supports the implemented Milestone 1 requirements, including registration and login, course CRUD, academic task CRUD, task progress and status updates, dashboard data, and user-specific data ownership.

Using REST and JSON provides a simple and consistent contract between the frontend

and backend and allows each part of the system to be developed and tested independently.

7.4 Alternatives Considered

One alternative was to store application data directly in the browser using local storage. This approach would have been simpler during early prototyping, but it would not provide secure authentication, reliable multi-user data isolation, or centralized persistent storage.

Another alternative was to combine the frontend and backend logic into a single tightly coupled application. This could reduce initial setup complexity, but it would make the system harder to maintain and would provide less separation between interface, application logic, and data access.

7.5 Consequences and Tradeoffs

The selected architecture improves separation of concerns, persistence, user isolation, and maintainability. It also makes backend APIs independently testable and provides a clearer path for future development.

The main tradeoff is additional implementation complexity because the frontend and backend must maintain compatible API contracts. Authentication tokens, request formats, response formats, and error handling must remain consistent across both sides of the application.

This tradeoff became visible during Milestone 1 integration, when frontend and backend field names, authentication handling, dashboard response structures, and delete-response behavior had to be aligned before the complete end-to-end workflow operated correctly.

8 Component Design

This section describes the component-level design of two important StudyFlow use cases: User Authentication and Academic Task Management. The designs show how frontend components, backend routes and services, authentication middleware, and the PostgreSQL database cooperate to complete each user interaction.

8.1 User Authentication

The authentication flow is responsible for user registration, login, authentication-token management, and access to protected resources.

The main components involved are:

- React authentication pages and authentication context
- `authService` in the frontend
- Express authentication routes
- authentication middleware
- PostgreSQL users table
- `bcrypt` password hashing
- JSON Web Token generation and verification

During registration, the frontend collects the user's name, email, password, and password confirmation. After frontend validation, the registration request sends the required account information to the backend. The backend validates the fields, normalizes the email address, checks for duplicate accounts, hashes the password using `bcrypt`, and stores the new user in PostgreSQL.

During login, the frontend sends the email and password to the backend. The backend retrieves the matching user record, compares the submitted password with the stored password hash, and generates a JWT when authentication succeeds. The frontend stores the authentication state and uses the token in the Authorization header for protected API requests.

For protected requests, the authentication middleware verifies the JWT and stores the

authenticated user ID in `req.userId`. Course, Task, and Dashboard routes use the authenticated identity to retrieve or modify only data belonging to the current user.

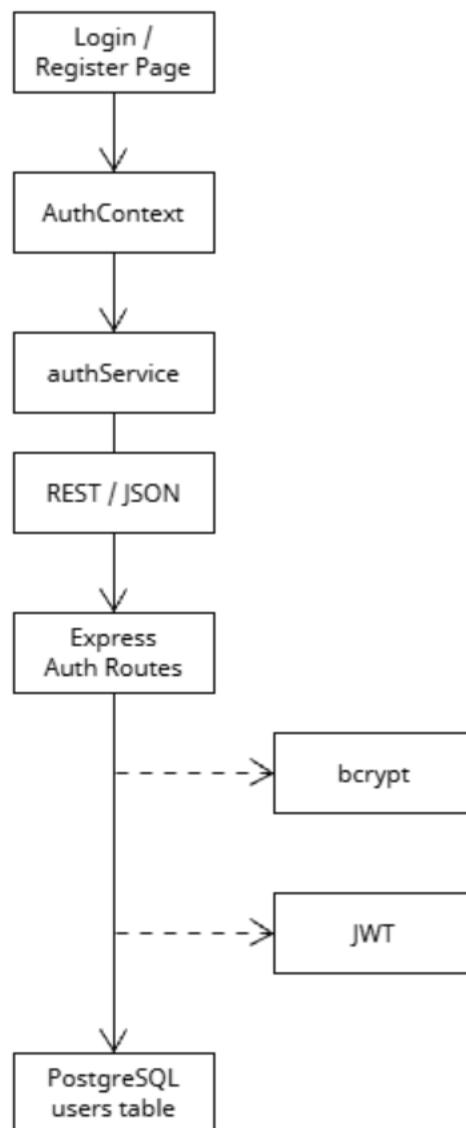


Figure 8.1. User authentication component diagram.

8.2 Academic Task Management

The Academic Task Management flow allows an authenticated user to create, view,

update, and delete academic tasks while ensuring that each user can access only their own data.

The main components involved are:

- React Task pages and forms
- `taskService` in the frontend
- Express Task routes
- authentication middleware
- PostgreSQL `academic_tasks` table
- PostgreSQL `courses` table

When a user creates a task, the frontend sends task information to the backend through the Task API. The request includes the authentication token. The authentication middleware validates the token and identifies the user.

The backend validates the required task fields, due date, progress value, status, estimated hours, and any optional course association. A task may be created without a course. If a course is provided, the backend verifies that the course belongs to the authenticated user before creating or updating the task.

For task retrieval, the backend queries only rows whose `user_id` matches `req.userId`. If a task is associated with a course, the associated course number can be returned for display in the frontend. Update and delete operations also include the authenticated user ID in the database conditions, preventing users from modifying tasks belonging to another account.

Task status and progress are kept consistent through normalization rules. When a task is marked as Completed, its progress is set to 100 percent. When progress reaches 100 percent, the task is normalized to the Completed state. When progress is between 1 and 99 percent, a task that is still marked Not Started is normalized to In Progress.

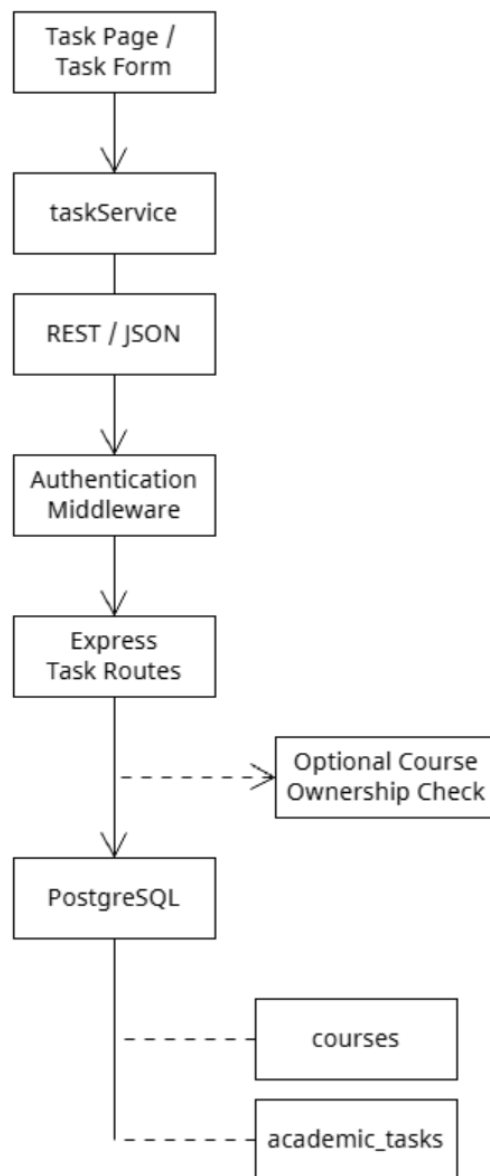


Figure 8.2. Academic Task Management component diagram.

9 UX Design and Interface

9.1 UX Design Approach

StudyFlow's user interface is designed to keep the Milestone 1 workflow simple and easy to follow for students managing courses and academic tasks.

- **Consistent Navigation:** A top navigation bar provides access to the Dashboard, Courses, and Academic Tasks views and reflects the current authentication state.
- **Card-Based Layouts:** Courses and academic tasks are displayed using card-based layouts. Course cards present the course number, while task cards display important information such as priority, progress, status, due date, and associated course when available.
- **Focused Forms and Feedback:** Course and task creation and editing use focused forms with validation and error feedback.
- **MVP-Focused Interface:** Features deferred beyond Milestone 1 are excluded from the interface so that the design remains focused on the required core workflow.

9.2 Low-Fidelity Wireframes

The following low-fidelity wireframes summarize the basic layout of the primary authenticated StudyFlow views. They show the placement of navigation, dashboard summary information, course cards, task cards, and the primary actions used to create and manage academic information.

DASHBOARD	Courses	ACADEMIC TASKS
Nav Bar	Nav Bar	Nav Bar
Summary Cards [Total] [Done]	+ Add Course [CS 415] [Edit] [Delete] [CS 470] [Edit] [Delete]	+ Add Task [Assignment] Due Date Priority / Progress Status [Edit] [Delete]
Task Card Course / Due Date Progress / Status		

Figure 9.1. Low-fidelity wireframes for the Dashboard, Courses, and Academic Tasks views.

9.3 Final Multi-Screen Interface

The completed Milestone 1 interface implements the core layouts established during UX design and supports the complete authenticated workflow across multiple application screens.

The screenshot displays the 'StudyFlow' login page. At the top left is the 'StudyFlow' logo, and at the top right are 'Login' and 'Register' links. The main content area features a white rounded rectangle with the title 'Sign in to StudyFlow'. Below the title is a link: 'Don't have an account? [Create an account](#)'. The form includes two input fields: 'Email Address' with the placeholder 'student@example.com' and 'Password' with masked characters '*****'. A blue 'Sign In' button is positioned at the bottom of the form.

Figure 9.2. Final Milestone 1 authentication interface.

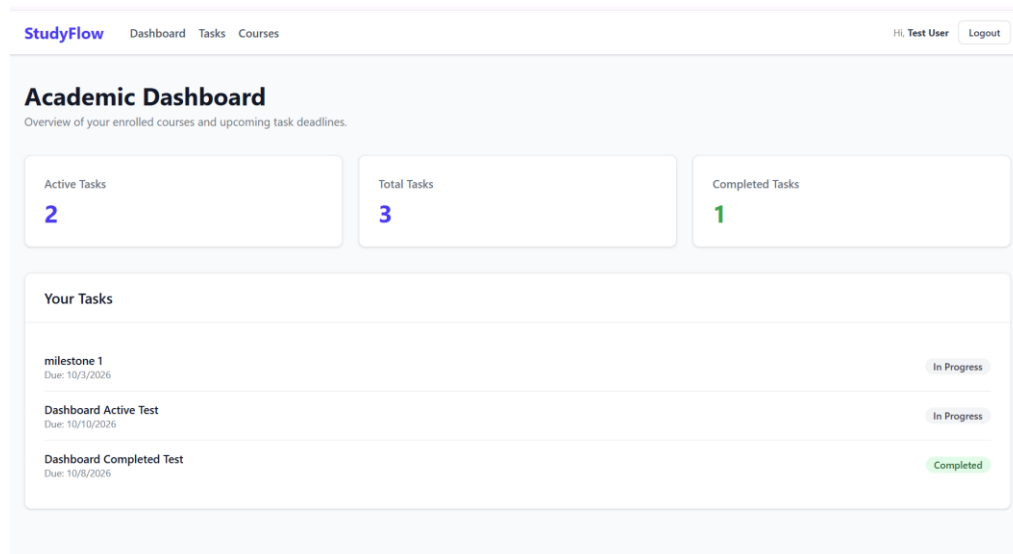


Figure 9.3. Final Milestone 1 dashboard showing task summaries and current task status.

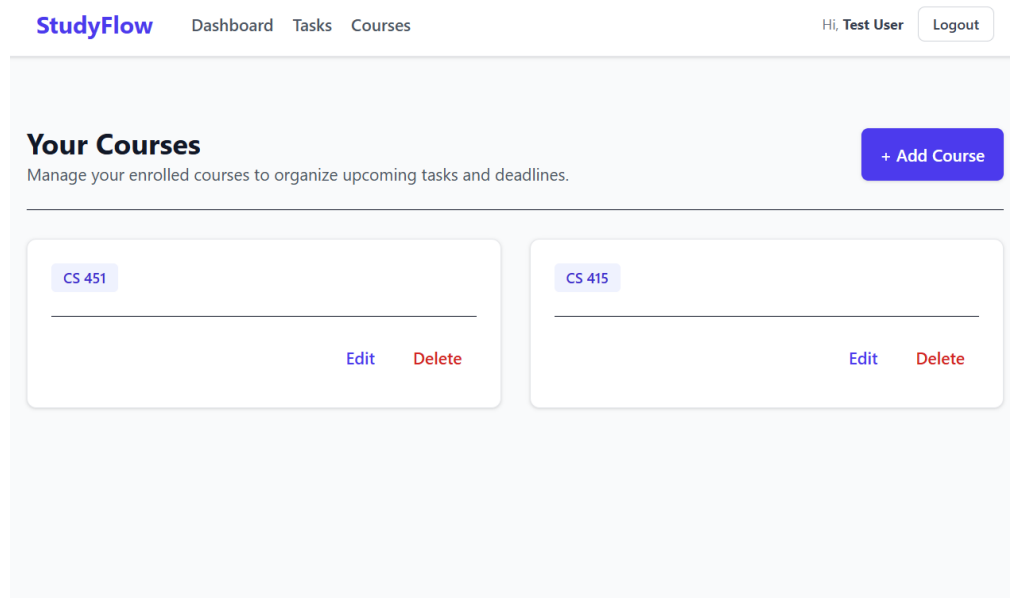


Figure 9.4. Final Milestone 1 course-management interface using course-number-based records.

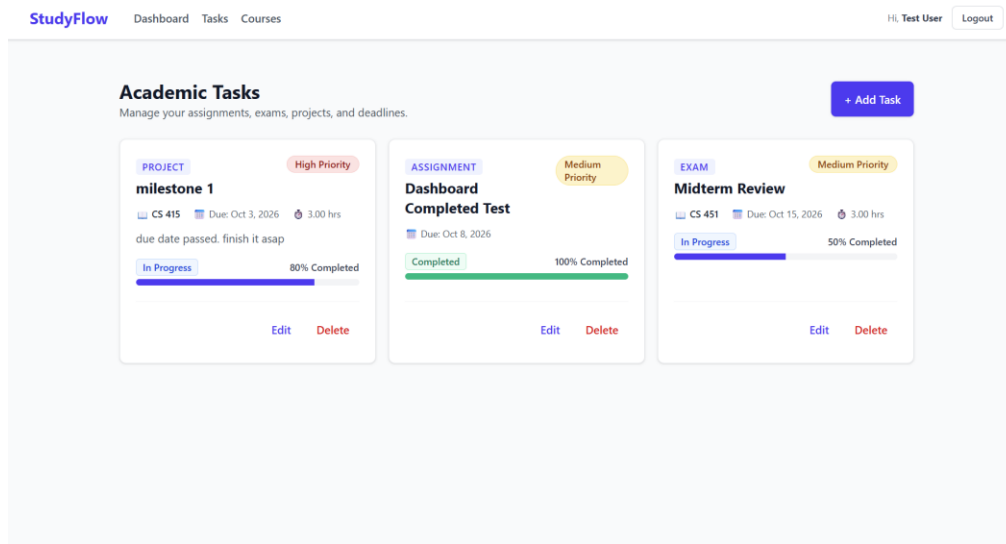


Figure 9.5. Final Milestone 1 academic-task management interface showing task details, progress, status, and course association.

9.4 Responsive Design Considerations

StudyFlow uses React and Tailwind CSS to support layouts that adapt to different screen sizes. Responsive grid and flexbox layouts allow dashboard content, course cards, task cards, navigation, forms, and dialogs to adjust or stack on smaller screens. Interactive controls use spacing and sizing intended to remain usable across desktop and smaller-screen layouts. Responsive behavior was verified after frontend/backend integration, including testing at a 390×844 mobile viewport.

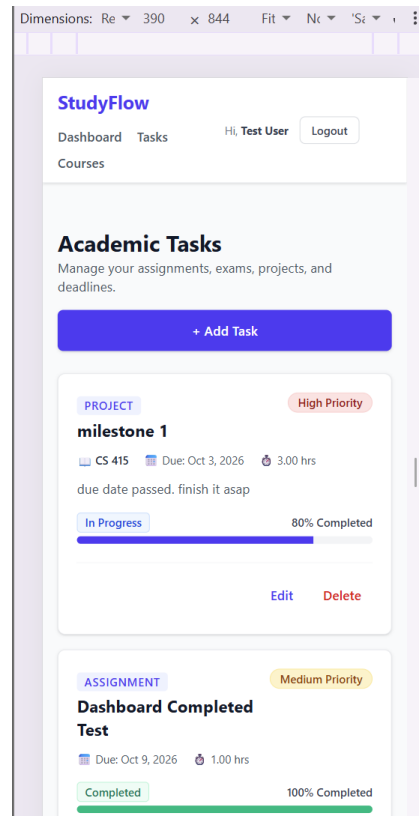


Figure 9.6. Final Milestone 1 mobile layout at a 390 × 844 viewport.

10 Design Patterns

StudyFlow uses lightweight design patterns to separate responsibilities and reduce duplication within the frontend. For Milestone 1, two patterns are particularly relevant: the Context Provider Pattern for shared authentication state and the Service Layer Pattern for communication with backend APIs.

10.1 Context Provider Pattern

The frontend uses React context to manage authentication state that must be shared across multiple components. The authentication context provides information such as the current user and authentication status, as well as actions for login, registration, and logout.

Without a shared context, individual pages and navigation components would need to manage or retrieve authentication state independently. Centralizing this state reduces duplication and helps keep authentication behavior consistent across protected pages and navigation elements.

In StudyFlow, this pattern is represented by the authentication context and the components that consume it, such as login, registration, navigation, and protected-route components.

A tradeoff is that components using the context depend on the provider being correctly configured near the top of the React component tree. If the provider is missing or authentication state is handled inconsistently outside the context, frontend behavior can become inconsistent.

In the final Milestone 1 implementation, `AuthProvider` is configured at the application level so that navigation, protected routes, and authenticated pages share the same authentication state.

10.2 Service Layer Pattern

StudyFlow separates backend API communication from page and UI components

through frontend service modules such as `authService`, `courseService`, `taskService`, and `dashboardService`.

These service modules are responsible for creating HTTP requests, attaching authentication tokens, interpreting API responses, and reporting request failures. UI components can therefore focus primarily on displaying data and responding to user interactions instead of duplicating networking logic.

This separation is useful because backend endpoint details and request handling can be updated in one location rather than across multiple pages. It also makes the frontend structure easier to maintain as the application grows.

The primary tradeoff is that the service layer and backend API must maintain compatible request and response contracts. Changes to field names, authentication handling, or response formats may require coordinated updates between the frontend services and backend routes.

This coordination was required during Milestone 1 integration when service payloads and backend response fields were aligned for Course, Task, Dashboard, and authentication flows.

11 Setup and Verification Summary

StudyFlow consists of a React frontend, a Node.js/Express backend, and a PostgreSQL database. The application requires the frontend and backend dependencies to be installed, the database to be initialized using the provided migrations, and the required environment variables to be configured before the complete application can be run. A detailed Windows installation and verification guide is provided in `docs/TA_WINDOWS_SETUP.md` in the project repository.

11.1 Backend and Database Setup

The backend is located in the server directory and uses Node.js and Express.

PostgreSQL is used for persistent application data.

To install the backend dependencies and start the development server, use:

```
cd server
npm install
npm run dev
```

The `npm run dev` command starts `src/server.js` with nodemon. The backend can also be started directly with:

```
npm start
```

Before starting the backend, create a local `.env` file based on the provided `.env.example`. The local environment must define `PORT`, `DATABASE_URL`, and `JWT_SECRET`. The `DATABASE_URL` should be configured for the local PostgreSQL installation, including the appropriate host, port, database name, username, and password.

After PostgreSQL is configured, create the StudyFlow database and apply the provided migrations in order:

```
001_create_users.sql
002_create_courses.sql
003_create_academic_tasks.sql
004_rename_course_name_to_course_number.sql
```

These migrations create the tables and relationships required for user accounts, courses, and academic tasks, and align the final Course schema with the courseNumber-based Milestone 1 model.

Environment-specific configuration is stored locally in the `.env` file and is excluded from version control. The provided `.env.example` documents the required configuration without exposing database credentials or authentication secrets.

The Milestone 1 setup was verified using Node.js v20.20.2, npm 10.8.2, and PostgreSQL 18.6.

No seed script or shared demonstration account is required; the TA can register a local account and create sample courses and academic tasks through the application.

11.2 Frontend Setup

The frontend is located in the client directory and is implemented using React with Vite and Tailwind CSS. Frontend dependencies must be installed before starting the development server.

To install the frontend dependencies and start the application in development mode, use:

```
cd client
npm install
npm run dev
```

The `npm run dev` command starts the Vite development server. A production build can be generated with:

```
npm run build
```

The frontend communicates with the Express backend through the StudyFlow

REST/JSON API. Authentication tokens are included in protected API requests so that backend routes can identify the authenticated user and enforce user-specific data ownership.

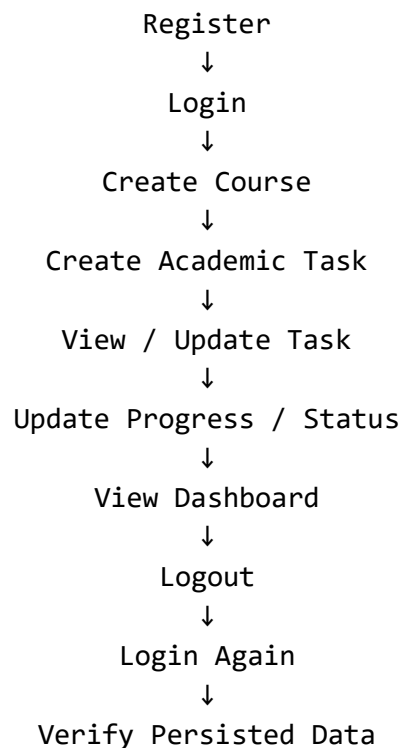
The final frontend/backend integration was verified after implementation.

Authentication, Course Management, Academic Task Management, Dashboard behavior, protected routes, persistence, and responsive behavior operate together as part of the completed Milestone 1 workflow.

11.3 Verification

Backend API behavior was manually verified for authentication, Course CRUD, Academic Task CRUD, user-specific data ownership, validation, and dashboard summary retrieval.

The final Milestone 1 end-to-end verification followed this workflow:



The final verification also confirmed protected-route behavior, Course and Task edit/delete operations, task progress/status normalization, Dashboard updates, persistence after refresh and re-login, user-specific data isolation, and responsive interface behavior on desktop and a 390×844 mobile viewport.

This workflow verified that the main MVP features operate together across the frontend, backend, and database rather than only as individually tested components. The setup process and complete end-to-end verification workflow were rechecked after frontend/backend integration was completed.

12 Contribution Statements

Eunjoo Jung completed the entire Milestone 1 backend implementation, including authentication, user-specific data ownership, Course and Academic Task CRUD, Dashboard support, PostgreSQL schema and migrations, validation, and task progress/status handling. After the frontend handoff, she also completed the remaining frontend corrections, frontend/backend integration, responsive-interface fixes, end-to-end verification, and final revision and alignment of the Milestone 1 report.

Errin James contributed to the frontend authentication, Dashboard, Course, and Academic Task interfaces and related frontend service code. She also drafted the Use Cases, Frontend Component Design, UX Design and Interface, and Frontend Setup portions of the report, which were later revised and integrated into the final submission.